PES University, Bengaluru

(Established under Karnataka Act 16 of 2013)

END SEMESTER ASSESSMENT (ESA) - Jan - May 2024

UE20CS353 - Compiler Design

Total Marks : 100.0

1.a. Explain with a diagram the various phases of a compiler. (8.0 Marks)

1.b. Explain in brief the role of a lexer. Is it possible to have a common lexer for all the
languages? Justify your answer with at least 2 examples. (4.0 Marks)

1.c. Construct transition diagrams for whitespaces and identifiers. (4.0 Marks)

1.d. i) What is Lex tool? Mention the sections of Lex specification program.
ii) Consider the following lexical specification:

```
%%
a*b             { printf(" 1 "); }
(a|b)*b         { printf(" 2 "); }
c*              { printf(" 3 "); }
```

Explain how lex would tokenize the following input string and the output produced.
cbbbbac (4.0 Marks)

2.a. Explain in detail Phrase-Level (Statement Mode) and Error Productions Error-Recovery Strategies. (4.0 Marks)

2.b. For the following grammar answer the following:
S → +SS | * SS | a

1. Construct first and follow table.
2. Construct LL(1) table and Mention whether the grammar is in LL(1) or not.
3. Show how to parse the input *+aa*

(6.0 Marks)

2.c. Consider the following grammar:
S → AB | a
A → a
B → b

1. Construct LR(0) automata
2. Construct SLR parsing table to check whether the given grammar is SLR or not.

(6.0 Marks)

2.d. i) Explain in brief Canonical LR(1) Items.
ii) Why is LR(1)/CLR(1) parser so powerful? (4.0 Marks)

3.a. Explain the following
i) Inherited Attributes
ii) L-attributed SDT
iii) Annotated parse tree
iv) Dependency graph (8.0 Marks)

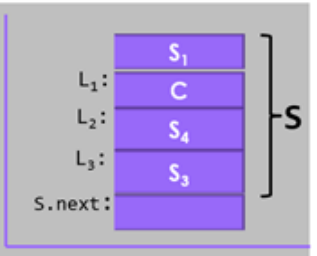
3.b. Consider the following arithmetic expression grammar.

$E \rightarrow E + T$
$E \rightarrow E - T$
$E \rightarrow T$
$T \rightarrow T * F$
$T \rightarrow T / F$
$T \rightarrow F$
$F \rightarrow num$

Write an SDD to convert a given infix expression to postfix expression during parsing. (4.0 Marks)

3.c. Consider the following SDD (to generate intermediate code) for for-statement

Production	Semantic Rules
$S \rightarrow for(S_1; C; S_3)S_4$	L1=new(); L2=new(); L3=new(); S_1.next=L1; C.true=L2; C.false=S.next; S_4.next=L3; S_3.next=L1; S.code=S_1.code||label||L1||C.code||label||L2||S_4.code ||label||L3||S_3.code



Let us turn this L-attributed SDD into an L-attributed SDT (i.e., suitable for implementation). (4.0 Marks)

3.d. Write syntax directed definition (SDD) to generate three-address intermediate code for given Conditional Boolean expression production.
$C \rightarrow B_1 \mid\mid B_2$
Note: Use function *new()* that generates new labels. (4.0 Marks)

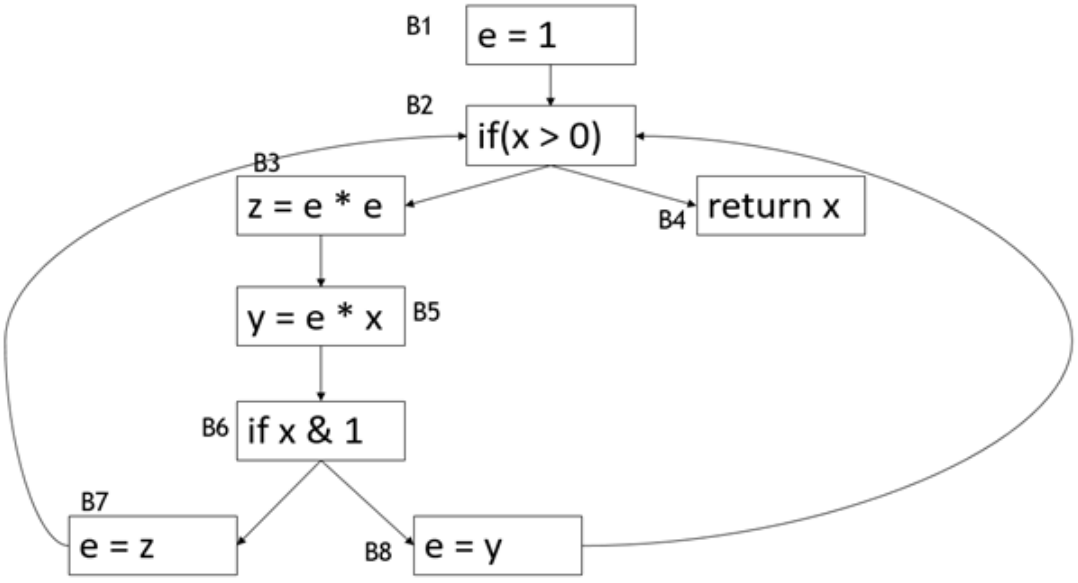
4.a. Write short notes on:
i) Quadruple
ii) Static Single Assignment (SSA) (5.0 Marks)

4.b. Construct control-flow graph (CFG) for the following three-address code (5.0 Marks)

```
    x = 0
    y = 0
L1: ifFalse x < 10 goto L2
    t1 = y + x;
    y = t1
    t2 = x + 1;
    x = t2
    goto L1
L2: print y
```

4.c. What is code optimization? Explain with an example any two code optimization techniques. (5.0 Marks)

4.d. Define use_B (or use[B]) and def_B (or def[B]).
Calculate use[B], def[B] for all the basic blocks on the following flow graph. (5.0 Marks)



5.a. Explain in brief any four issues in the design of a code generator. (4.0 Marks)

5.b. Explain the following in brief
i) Activation record
ii) Activation tree
iii) Calling Sequence and Return sequence (6.0 Marks)

5.c. Explain the following in brief
i) Access Links
ii) Displays (4.0 Marks)

5.d. Generate target code sequence for the following basic block three address statements with register and address descriptors.

```
1.  x:= y-z
2.  z:= x*x
3.  y:= z
4.  x:= y-z
```

Note 1: Assuming only two registers (R1 and R2) are available and all the variables are live on exit from the basic block. No temporary variables are used.
Note 2: Assuming that all arithmetic operations take their operands from registers.
(6.0 Marks)